CESGA Supercomputer Simulator — Guía de Usuario

> ### 🌐 API desplegada — acceso público
>
> **URL pública:** `https://api-mock-cesga.onrender.com`
> **Documentación interactiva (Swagger):** `https://api-mock-cesga.onrender.com/docs`
> **Sin autenticación requerida** — todos los endpoints son públicos
>
> > **Nota cold-start:** La API está en el tier gratuito de Render. Si lleva más de 15 minutos sin recibir peticiones, el primer request puede tardar ~30 segundos en responder mientras el servidor despierta. Los siguientes son instantáneos.

Acceso rápido — cómo hacer tu primera petición

La forma más rápida de probar la API sin instalar nada es desde el navegador o con `curl`. No necesitas cuenta ni token.

Desde el navegador

Abre directamente:

`https://api-mock-cesga.onrender.com/docs`

Verás la interfaz Swagger — cada endpoint tiene un botón **"Try it out"** que permite ejecutar peticiones sin escribir código.

Desde la terminal (curl)

1. Verificar que la API está activa:

```
curl https://api-mock-cesga.onrender.com/health
```

Respuesta esperada:

```
{"status":"healthy","service":"CESGA Supercomputer Simulator","version":"
```

2. Ver el catálogo de proteínas disponibles:

```
curl https://api-mock-cesga.onrender.com/proteins/
```

3. Obtener secuencias FASTA de ejemplo listas para usar:

```
curl https://api-mock-cesga.onrender.com/proteins/samples
```

4. Enviar un job de predicción de estructura:

```
curl -X POST https://api-mock-cesga.onrender.com/jobs/submit \
  -H "Content-Type: application/json" \
  -d '{
    "fasta_sequence": ">sp|P0CG47|UBQ_HUMAN Ubiquitin\nMQIFVKTLTGKTITLEVE
    "fasta_filename": "ubiquitin.fasta",
    "gpus": 1,
    "memory_gb": 16
  }'
```

Respuesta:

```
{"job_id":"job_xxxxxxxxxxxx","status":"PENDING","message":"Job submitted
```

5. Consultar el estado del job (sustituye `JOB_ID` por el que te devolvió el paso anterior):

```
curl https://api-mock-cesga.onrender.com/jobs/JOB_ID/status
```

6. Obtener los resultados (una vez que el estado sea `COMPLETED`, ~5-10 segundos):

```
curl https://api-mock-cesga.onrender.com/jobs/JOB_ID/outputs
```

7. Ver la contabilidad de recursos HPC:

```
curl https://api-mock-cesga.onrender.com/jobs/JOB_ID/accounting
```

Flujo completo en un script bash

```bash
#!/bin/bash
BASE="https://api-mock-cesga.onrender.com"

# 1. Enviar job
echo "Enviando job..."
RESPONSE=$(curl -s -X POST "$BASE/jobs/submit" \
  -H "Content-Type: application/json" \
  -d '{
    "fasta_sequence": ">sp|P0CG47|UBQ_HUMAN Ubiquitin\nMQIFVKTLTGKTITLEVE
    "fasta_filename": "ubiquitin.fasta",
    "gpus": 1,
    "memory_gb": 16
  }')
JOB_ID=$(echo "$RESPONSE" | python3 -c "import sys,json; print(json.load(
echo "Job ID: $JOB_ID"

# 2. Esperar hasta COMPLETED
STATUS="PENDING"
while [ "$STATUS" != "COMPLETED" ] && [ "$STATUS" != "FAILED" ]; do
  sleep 3
  STATUS=$(curl -s "$BASE/jobs/$JOB_ID/status" | python3 -c "import sys,j
  echo "Estado: $STATUS"
done

# 3. Obtener resultados
echo "Outputs:"
curl -s "$BASE/jobs/$JOB_ID/outputs" | python3 -m json.tool | head -40

echo "Accounting:"
curl -s "$BASE/jobs/$JOB_ID/accounting" | python3 -m json.tool
```

Desde Python

```python
import requests
import time

BASE = "https://api-mock-cesga.onrender.com"

# 1. Enviar job
```

```
resp = requests.post(f"{BASE}/jobs/submit", json={
    "fasta_sequence": ">sp|P0CG47|UBQ_HUMAN Ubiquitin\nMQIFVKTLTGKTITLEVE
    "fasta_filename": "ubiquitin.fasta",
    "gpus": 1,
    "memory_gb": 16
})
job_id = resp.json()["job_id"]
print(f"Job enviado: {job_id}")

# 2. Polling
while True:
    status = requests.get(f"{BASE}/jobs/{job_id}/status").json()["status"
    print(f"Estado: {status}")
    if status in ("COMPLETED", "FAILED", "CANCELLED"):
        break
    time.sleep(3)

# 3. Resultados
if status == "COMPLETED":
    outputs = requests.get(f"{BASE}/jobs/{job_id}/outputs").json()
    plddt   = outputs["structural_data"]["confidence"]["plddt_mean"]
    sol     = outputs["biological_data"]["solubility_score"]
    meta    = outputs.get("protein_metadata")
    print(f"pLDDT medio: {plddt:.1f}")
    print(f"Solubilidad: {sol:.1f}/100")
    if meta:
        print(f"Proteína identificada: {meta['protein_name']} ({meta['uni

    # Guardar el fichero PDB
    pdb_content = outputs["structural_data"]["pdb_file"]
    with open(f"{job_id}.pdb", "w") as f:
        f.write(pdb_content)
    print(f"Estructura guardada en {job_id}.pdb")

    # Contabilidad
    acc = requests.get(f"{BASE}/jobs/{job_id}/accounting").json()
    print(f"CPU-hours: {acc['accounting']['cpu_hours']:.4f}")
```

> ## ⚠️ AVISO IMPORTANTE — Limitación actual de la base de datos

Las proteínas incluidas en esta API son secuencias de prueba, no una base de datos de producción.

El catálogo actual contiene **22 proteínas curadas manualmente** (ubiquitina, calmodulina, GFP, p53, etc.). Estas secuencias están aquí únicamente para que puedas desarrollar y testear tu frontend sin necesitar acceso real al CESGA.

¿Qué pasa si envías una secuencia que no está en el catálogo?

La API **acepta cualquier secuencia FASTA válida** — no hay restricción técnica. Sin embargo, el comportamiento cambia:

Secuencia	Resultado
Está en las 22 del catálogo	Identificación automática + metadata real (UniProt, PDB, organismo) + estructura precomputada
No está en el catálogo	Se procesa igualmente, pero `protein_metadata` será `null` y toda la estructura y datos biológicos serán **sintéticos** (generados algorítmicamente, no basados en datos reales de esa proteína)

En resumen: puedes meter cualquier cadena de aminoácidos, pero solo obtendrás datos biológicos reales para las proteínas que están definidas en el sistema.

Para qué sirve esta API y cuál es el objetivo real

Esta API es un simulador, no el sistema final. Existe para que puedas construir y probar el frontend sin necesitar acceso real al CESGA ni tener AlphaFold instalado. Devuelve los mismos campos y la misma estructura de datos que devolvería el sistema real — solo que los cálculos son simulados en lugar de ejecutarse en el supercomputador.

El objetivo real es construir un **portal web completo de predicción de estructuras proteicas**, conectado al CESGA Finis Terrae III, que permita a investigadores sin experiencia en HPC usar AlphaFold2 con la misma facilidad con que usan la AlphaFold Database pública.

El problema que resuelve: Las herramientas de predicción de estructuras como AlphaFold2 son potentes pero inaccesibles para la mayoría de investigadores. Una instalación completa requiere Linux, una GPU NVIDIA moderna, y hasta ~3 TB de disco solo para las bases de datos genéticas. Centralizar eso en el CESGA y exponerlo a través de una interfaz web es la solución — pero alguien tiene que construir esa interfaz.

Cómo sería el sistema completo:

La arquitectura tiene tres capas:

```
Investigador (navegador)
      ↓  pega secuencia FASTA
Frontend web  (React / SPA)
      ↓  POST /jobs/submit
Backend API  (Python, base de datos de jobs, gestión de ficheros)
      ↓  sbatch → Slurm
CESGA Finis Terrae III  (nodos GPU NVIDIA A100, Apptainer,
AlphaFold2)
      ↓  outputs: PDB, mmCIF, pLDDT.json, PAE.json
Backend API recoge resultados → Frontend los visualiza
```

Lo que el usuario vería en el portal real:

1. **Entrada**: un campo de texto que acepta cualquier secuencia FASTA (de UniProt, de laboratorio, de un paper), sin restricciones de catálogo. Presets simples: "Vista rápida", "Estándar", "Alta precisión".
2. **Estado del job**: página clara con PENDIENTE / EJECUTANDO / COMPLETADO / FALLIDO, con logs legibles y descargables.
3. **Resultados interactivos**:
   - Visor 3D molecular embebido (rotación, zoom, selección de residuos) usando **Mol*** — el mismo visor que usa la AlphaFold Database oficial de EMBL-EBI.
   - Estructura coloreada por confianza pLDDT (azul = muy confiable, naranja = región probablemente desordenada).
   - Heatmap 2D de la matriz PAE: bloques azules en la diagonal indican dominios bien definidos; bloques amarillos entre dominios indican orientación relativa incierta.
   - Panel resumen: pLDDT medio, fracción de residuos por rango de confianza, avisos si hay regiones de baja confianza.
   - Botones de descarga: PDB, mmCIF, JSON de confianza, logs completos.

Integración con CESGA:

El backend generaría scripts Slurm (`sbatch`) automáticamente, solicitaría GPUs con `--gres=gpu` , monitorearía el estado con `squeue` / `sacct` , y ejecutaría AlphaFold2 dentro de un contenedor Apptainer (el equivalente HPC de Docker). Los datos de secuencia no saldrían a servicios externos — todo se procesa dentro del CESGA, lo cual es importante para secuencias de investigación no publicadas.

Engines de predicción que podría soportar:

- **AlphaFold2** — precisión state-of-the-art, requiere las bases de datos genéticas completas (~3 TB).
- **ColabFold** — misma arquitectura que AlphaFold2 pero con búsqueda de homólogos acelerada, bases de datos más ligeras. Muy usado en entornos HPC por su menor huella

de almacenamiento.
- **RoseTTAFold** — arquitectura alternativa de Rosetta; código MIT pero pesos del modelo de uso no comercial exclusivamente.

Lo que esta API ya tiene listo para ese sistema:

La estructura de datos que devuelve esta API (`pdb_file` , `plddt_per_residue` , `pae_matrix` , `biological_data` , `accounting`) es exactamente la que necesitaría consumir el frontend del sistema real. Si construyes el visor 3D, el heatmap de PAE y el panel de confianza sobre esta API, funcionarán sin cambios cuando se conecten al CESGA real — solo cambia el origen de los datos (base de datos local simulada → AlphaFold2 ejecutando en nodos A100).

Lo que faltaría implementar para el sistema real:

1. Autenticación con cuentas CESGA o SSO institucional.
2. Integración real con Slurm: generación de scripts `sbatch` , polling de `squeue` / `sacct` , gestión de errores de cola.
3. Contenedor Apptainer con AlphaFold2 y las bases de datos genéticas en almacenamiento compartido del CESGA.
4. Almacenamiento persistente de resultados (sistema de ficheros Lustre del CESGA o equivalente).
5. Panel de administración para monitorizar jobs, uso de GPUs y cuotas por usuario.
6. Política de retención de datos y confidencialidad (las secuencias de investigación no deben salir del clúster).

Conceptos previos

Si ya conoces la biología molecular y el HPC puedes saltar directamente al Índice. Esta sección es para quien llega desde el lado de software y necesita entender el contexto.

¿Qué es una proteína?

Las proteínas son las **máquinas moleculares** de los seres vivos. Casi todo lo que ocurre en una célula — digerir comida, mover músculos, defenderse de infecciones, copiar el ADN— lo ejecuta alguna proteína.

Estructuralmente, una proteína es una cadena larga formada por la unión en serie de piezas más pequeñas llamadas **aminoácidos**. Hay 20 aminoácidos diferentes (como un alfabeto de 20 letras), y su orden concreto en la cadena es lo que determina qué proteína es y qué hace. Una proteína típica tiene entre 50 y 1000 aminoácidos encadenados.

```
Met - Gln - Ile - Phe - Val - Lys - Thr - Leu - Thr - Gly - ...
 M  -  Q  -  I  -  F  -  V  -  K  -  T  -  L  -  T  -  G  - ...
```

Una vez sintetizada, esta cadena **se pliega** espontáneamente en una forma tridimensional específica (como origami molecular). Esa forma 3D es lo que determina su función: una enzima tiene un hueco concreto donde encaja su sustrato, un receptor tiene una superficie exacta donde se une su hormona, etc.

El problema fundamental: conocer la secuencia de aminoácidos (el "texto" de la proteína) es relativamente fácil y barato desde los años 90. Pero conocer la estructura 3D que adopta esa secuencia era hasta hace poco lento, caro y difícil —requería cristalografía de rayos X o resonancia magnética nuclear, ambas técnicas de laboratorio muy exigentes. De ahí el valor de la predicción computacional.

¿Qué es una secuencia FASTA?

FASTA es el **formato de texto estándar** para representar secuencias biológicas (tanto proteínas como ADN). Es el equivalente al `.txt` del mundo bioinformático.

Su estructura es muy sencilla:

```
>identificador descripción opcional
SECUENCIA_DE_AMINOÁCIDOS_O_NUCLEÓTIDOS
```

- La primera línea empieza siempre con `>` y es el **header**: contiene un ID y opcionalmente metadatos.
- Las líneas siguientes son la **secuencia**, usando el código de una letra por aminoácido.

Ejemplo real — Ubiquitina humana:

```
>sp|P0CG47|UBQ_HUMAN Ubiquitin OS=Homo sapiens
MQIFVKTLTGKTITLEVEPSDTIENVKAKIQDKEGIPPDQQRLIFAGKQLEDGRTLSDYNIQKESTLHLVLRLR
GG
```

El código de una letra por aminoácido es:

Letra	Aminoácido	Letra	Aminoácido	Letra	Aminoácido
A	Alanine	H	Histidine	R	Arginine
C	Cysteine	I	Isoleucine	S	Serine
D	Aspartate	K	Lysine	T	Threonine
E	Glutamate	L	Leucine	V	Valine
F	Phenylalanine	M	Methionine	W	Tryptophan

Letra	Aminoácido	Letra	Aminoácido	Letra	Aminoácido
G	Glycine	N	Asparagine	Y	Tyrosine
Q	Glutamine	P	Proline		

El formato FASTA se usa universalmente: bases de datos como **UniProt** (proteínas) y **GenBank** (genes) entregan sus secuencias en este formato. Es lo que esta API acepta como entrada.

¿Qué es AlphaFold?

AlphaFold es un sistema de inteligencia artificial desarrollado por **DeepMind** (Google) que resolvió uno de los grandes problemas abiertos de la biología: predecir la estructura 3D de una proteína a partir de su secuencia de aminoácidos.

Antes de AlphaFold, determinar la estructura de una proteína podía llevar años de trabajo de laboratorio. AlphaFold2 (2020) logró hacerlo en minutos con una precisión comparable a los métodos experimentales, ganando el primer puesto en la competición CASP14 por un margen histórico. En 2022, DeepMind publicó las estructuras predichas de casi **todas las proteínas conocidas** (~200 millones), disponibles en la AlphaFold Protein Structure Database.

¿Cómo funciona? AlphaFold2 combina dos etapas:

1. **Búsqueda de homólogos (MSA):** compara tu secuencia contra millones de secuencias evolutivamente relacionadas. Las posiciones que cambian juntas a lo largo de la evolución revelan qué residuos están cerca en el espacio 3D.
2. **Red neuronal de predicción estructural:** un modelo tipo Transformer que aprende patrones de cómo las secuencias se pliegan, entrenado sobre todas las estructuras conocidas del PDB.

Output principal: un fichero PDB o mmCIF con las coordenadas 3D de cada átomo, más un valor de confianza por residuo llamado **pLDDT** y una matriz de error llamada **PAE** (ver sección 6).

Esta API **simula** ese pipeline: acepta tu FASTA, lanza un "job" que imita el sistema de colas del supercomputador, y devuelve datos de estructura y confianza en el mismo formato que devolvería AlphaFold2 real.

¿Qué es el CESGA y Finis Terrae III?

El **CESGA** (Centro de Supercomputación de Galicia) es la infraestructura de computación de altas prestaciones de Galicia, con sede en Santiago de Compostela. Da servicio a universidades, centros de investigación y empresas de la región para proyectos que requieren más recursos que un ordenador convencional.

Finis Terrae III (FT3) es su supercomputador de última generación, equipado con nodos GPU basados en NVIDIA A100. Es el sistema que se usa realmente para correr AlphaFold2 en producción en el entorno académico gallego.

```
┌──────────────────────────────────────────────────────────┐
│                    CESGA FT3                             │
│                                                          │
│  Nodos CPU  ──┐                                          │
│  Nodos GPU  ──┼──  Sistema de colas SLURM                │
│  Storage    ──┘      (sbatch / squeue / sacct)          │
│                                                          │
│  Lustre filesystem (almacenamiento paralelo)             │
│  Infiniband network (comunicación entre nodos)           │
└──────────────────────────────────────────────────────────┘
```

¿Qué es Slurm? Es el gestor de colas de trabajos que usa el CESGA (y la mayoría de supercomputadores del mundo). Cuando envías un job, Slurm lo encola, espera a que haya recursos libres (GPUs, CPUs, memoria), lo ejecuta, y guarda los logs. Los comandos clave son:

- `sbatch script.sh` — enviar un job
- `squeue -u usuario` — ver mis jobs en cola
- `sacct -j jobid` — ver estadísticas de un job terminado

Apptainer (antes Singularity) es la tecnología de contenedores que usa el CESGA para ejecutar herramientas como AlphaFold2. En HPC no se puede usar Docker (requiere privilegios de root), así que Apptainer permite empaquetar toda la instalación de AlphaFold2 (modelo, dependencias, librerías) en un fichero `.sif` portátil que cualquier usuario puede ejecutar sin permisos especiales. Los logs que genera esta API simulan exactamente la salida que produce Apptainer al arrancar ese contenedor.

¿Qué simula exactamente esta API?

Esta API reproduce el flujo completo que experimentaría un usuario real del CESGA que quiere predecir estructuras proteicas:

```
Usuario real en CESGA:                    Esta API (simulador):
──────────────────────────────────────────────────────────────────────────────
1. Prepara fichero FASTA                  POST /jobs/submit  con fasta_sequence
2. Escribe script Slurm (sbatch)          → API genera job_id automáticamente
3. Lanza: sbatch alphafold_job.sh         → status: PENDING
```

```
4. Espera en cola (minutos/horas)          → status: PENDING (~5 s)
5. Job ejecutándose en nodo GPU            → status: RUNNING (~5-10 s)
6. Job termina, ficheros en $LUSTRE        → status: COMPLETED
7. Descarga structure.pdb, confidence.json → GET /jobs/{id}/outputs
8. Consulta sacct para ver CPU/GPU usados → GET /jobs/{id}/accounting
```

La diferencia es que en el simulador todo ocurre en segundos y sin necesitar cuenta en el CESGA. Los datos de estructura son sintéticos (para proteínas conocidas usa estructuras reales de la PDB; para el resto genera coordenadas simuladas), pero el formato de la respuesta, los campos, y el flujo son idénticos a lo que esperaría un frontend real.

Índice

0. Acceso rápido
1. Conceptos previos
2. Endpoints de Jobs
3. Endpoints de Catálogo de Proteínas
4. Campos de request y response explicados
5. Catálogo de proteínas disponibles
6. Secuencias FASTA listas para copiar
7. Entendiendo los outputs: pLDDT y PAE
8. Errores comunes y cómo resolverlos
9. Flujo completo de ejemplo

> Los ejemplos de código en las secciones siguientes usan `http://localhost:8000` (desarrollo local).
> Para la API pública, sustituye por `https://api-mock-cesga.onrender.com`.

---

1. Endpoints de Jobs

`POST /jobs/submit` — Enviar un job

Envía una secuencia proteica en formato FASTA para predicción de estructura.

Request body (JSON):

Campo	Tipo	Requerido	Default	Límites	Descripción

Campo	Tipo	Requerido	Default	Límites	Descripción
`fasta_sequence`	string	✅	—	1–100.000 chars	Secuencia en formato FASTA (debe empezar con `>`)
`fasta_filename`	string	✅	—	1–255 chars	Nombre del fichero (e.g. `mi_proteina.fasta`)
`gpus`	int	❌	0	0–4	Número de GPUs solicitadas
`cpus`	int	❌	1	1–64	Número de CPUs solicitadas
`memory_gb`	float	❌	4.0	0–256	Memoria RAM en GB
`max_runtime_seconds`	int	❌	3600	60–86400	Tiempo máximo de ejecución (1 min a 24 h)

Ejemplo:

```
curl -X POST http://localhost:8000/jobs/submit \
  -H "Content-Type: application/json" \
  -d '{
    "fasta_sequence": ">sp|P0CG47|UBQ_HUMAN Ubiquitin\nMQIFVKTLTGKTITLEVE
    "fasta_filename": "ubiquitin.fasta",
    "gpus": 1,
    "cpus": 8,
    "memory_gb": 32.0,
    "max_runtime_seconds": 3600
  }'
```

Response (201 Created):

```
{
  "job_id": "job_c5d06eb83abf",
  "status": "PENDING",
  "message": "Job submitted successfully. Use the job_id to check status.
}
```

`GET /jobs/{job_id}/status` — Estado del job

Consulta el estado actual de un job. Llama a este endpoint periódicamente hasta ver `"COMPLETED"`.

Ejemplo:

```
curl http://localhost:8000/jobs/job_c5d06eb83abf/status
```

Response:

```
{
  "job_id": "job_c5d06eb83abf",
  "status": "RUNNING",
  "created_at": "2026-04-09T12:00:24.386319",
  "started_at": "2026-04-09T12:00:29.931088",
  "completed_at": null,
  "gpus": 1,
  "cpus": 8,
  "memory_gb": 32.0,
  "max_runtime_seconds": 3600,
  "fasta_filename": "ubiquitin.fasta",
  "error_message": null
}
```

Estados posibles:

Status	Significado	¿Cuándo ocurre?
`PENDING`	En cola, esperando recursos	Primeros ~5 s tras submit
`RUNNING`	Ejecutándose en el clúster	Segundos 5–10
`COMPLETED`	Terminado con éxito, outputs disponibles	A partir de ~10 s
`FAILED`	Error durante la ejecución	Si la secuencia es inválida o hay error interno
`CANCELLED`	Cancelado por el usuario	No implementado en simulador

`GET /jobs/{job_id}/outputs` — Resultados del job

Devuelve los ficheros de estructura, confianza y datos biológicos. **Solo disponible cuando** `status = COMPLETED`.

Ejemplo:

```
curl http://localhost:8000/jobs/job_c5d06eb83abf/outputs
```

Response (200 OK):


```
{
  "job_id": "job_c5d06eb83abf",
  "status": "COMPLETED",

  "protein_metadata": {
    "identified_protein": "ubiquitin",
    "uniprot_id": "P0CG47",
    "pdb_id": "1UBQ",
    "protein_name": "Ubiquitin",
    "organism": "Homo sapiens",
    "description": "Small regulatory protein...",
    "data_source": "precomputed_database"
  },

  "structural_data": {
    "pdb_file": "HEADER    UBIQUITIN ...\nATOM      1  N   ...",
    "cif_file": "data_simulated_structure\n...",
    "confidence": {
      "plddt_per_residue": [72.3, 85.1, 91.4, ...],
      "plddt_mean": 71.7,
      "plddt_histogram": {
        "very_high": 13,
        "high": 29,
        "medium": 24,
        "low": 10
      },
      "pae_matrix": [[0.12, 1.8, ...], ...],
      "mean_pae": 3.33
    }
  },

  "biological_data": {
    "solubility_score": 79.8,
    "solubility_prediction": "soluble",
    "instability_index": 19.8,
    "stability_status": "stable",
    "toxicity_alerts": ["Potential signal peptide detected"],
    "allergenicity_alerts": [],
    "secondary_structure_prediction": {
      "helix_percent": 23.7,
      "strand_percent": 14.5,
      "coil_percent": 61.8
```

```
    },
    "sequence_properties": {
      "length": 76,
      "molecular_weight_kda": 8.4,
      "positive_charges": 11,
      "negative_charges": 11,
      "cysteine_residues": 0,
      "aromatic_residues": 3
    }
  },

  "logs": "[2026-04-09T14:00:35] Job started in Apptainer container\n[INF
}
```


Nota: Si llamas a este endpoint antes de que el job esté completado recibirás:


```
{
  "detail": "Job outputs only available when status=COMPLETED. Current st
}
```

`GET /jobs/{job_id}/accounting` — Contabilidad de recursos HPC

Devuelve el consumo ficticio pero realista de recursos computacionales. Disponible para cualquier job, independientemente del estado.

Ejemplo:

```
curl http://localhost:8000/jobs/job_c5d06eb83abf/accounting
```

Response:


```
{
  "job_id": "job_c5d06eb83abf",
  "status": "COMPLETED",
  "accounting": {
    "cpu_hours": 0.0074,
    "gpu_hours": 0.0012,
    "memory_gb_hours": 0.037,
    "total_wall_time_seconds": 5,
```

```
    "cpu_efficiency_percent": 66.9,
    "memory_efficiency_percent": 81.6,
    "gpu_efficiency_percent": 88.9
  }
}
```

`GET /jobs/` — Listar todos los jobs

Devuelve todos los jobs enviados, con paginación opcional.

Parámetros de query:

Parámetro	Default	Descripción
`skip`	0	Jobs a saltar (offset)
`limit`	100	Máximo de jobs a devolver

Ejemplo:

```
# Primeros 10 jobs
curl "http://localhost:8000/jobs/?limit=10"

# Jobs del 20 al 30
curl "http://localhost:8000/jobs/?skip=20&limit=10"
```

`GET /health` — Estado de la API

```
curl http://localhost:8000/health
# {"status": "healthy", "service": "CESGA Supercomputer Simulator", "vers
```

---

2. Endpoints de Catálogo de Proteínas

Estos endpoints permiten explorar la base de datos de proteínas conocidas y obtener las secuencias exactas para enviar.

`GET /proteins/` — Listar proteínas

Lista todas las proteínas identificables, con filtros opcionales.

Parámetros de query:

Parámetro	Descripción	Ejemplo
`category`	Filtra por categoría funcional	`enzyme`, `transport`, `signaling`, `immune`, `hormone`, `reporter`, `structural`, `oncology`, `dna-replication`
`search`	Busca por nombre, organismo o tag	`calcium`, `human`, `fluorescent`
`min_length`	Longitud mínima en aa	`50`
`max_length`	Longitud máxima en aa	`200`

Ejemplos:

```
# Todas las proteínas
curl http://localhost:8000/proteins/

# Solo enzimas
curl "http://localhost:8000/proteins/?category=enzyme"

# Proteínas pequeñas (< 150 aa)
curl "http://localhost:8000/proteins/?max_length=150"

# Buscar proteínas relacionadas con calcio
curl "http://localhost:8000/proteins/?search=calcium"

# Enzimas humanas entre 100 y 200 aa
curl "http://localhost:8000/proteins/?category=enzyme&min_length=100&max_
```

`GET /proteins/stats` — Estadísticas de la base de datos

```
curl http://localhost:8000/proteins/stats
```

```
{
  "total_proteins": 1022,
  "embedded_proteins": 22,
  "extended_proteins": 1000,
  "average_length": 278.0,
```

```
    "min_length": 76,
    "max_length": 585,
    "by_category": {
      "enzyme": 8,
      "transport": 4,
      "signaling": 3,
      "oncology": 2,
      ...
    }
  }
```

`GET /proteins/samples` — Secuencias de ejemplo listas para usar

Devuelve 8 secuencias FASTA de proteínas icónicas, listas para copiar y pegar.

```
curl http://localhost:8000/proteins/samples
```

`GET /proteins/{protein_id}` — Detalles de una proteína

Devuelve toda la información de una proteína, incluyendo la secuencia FASTA lista para enviar.

```
curl http://localhost:8000/proteins/calmodulin
curl http://localhost:8000/proteins/gfp
curl http://localhost:8000/proteins/sod1
```

Response incluye `fasta_ready` — cópialo directamente como `fasta_sequence` en el submit.

---

3. Campos de request y response explicados

`fasta_sequence` — Formato correcto

El campo FASTA debe tener **obligatoriamente** un header (`>`) seguido de la secuencia:

```
>identificador_opcional descripcion_opcional
MKFSMVQTLTGK...
```

✅ **Correcto:**

```
>mi_proteina
MQIFVKTLTGKTITLEVEPSDTIENK
```

✅ **También correcto (multilínea):**

```
>sp|P0CG47|UBQ_HUMAN Ubiquitin
MQIFVKTLTGKTITL
EVEPSDTIENVKAKI
QDKEGIPPDQQRLIF
```

❌ **Incorrecto (sin header):**

```
MQIFVKTLTGKTITLEVEPSDTIENVKAK
```

Identificación automática de proteínas

Cuando envías una secuencia, la API compara contra su base de datos de **22 proteínas bien caracterizadas**. Si hay coincidencia:

- `protein_metadata` en los outputs tendrá `uniprot_id`, `pdb_id`, `organism`, `description`
- `data_source` será `"precomputed_database"`

Si no hay coincidencia, la predicción es 100% sintética:

- `protein_metadata` será `null`
- `data_source` en `biological_data` será `"synthetic_prediction"`

Para garantizar la identificación, usa las secuencias del apartado [5. Secuencias FASTA]() o las del endpoint `GET /proteins/{id}`.

Recursos de hardware — guía rápida

Tipo de job	GPUs	CPUs	Memoria
Proteína pequeña (<100 aa), sin GPU	0	4	8 GB
Proteína mediana, estándar	1	8	32 GB
Proteína grande (>300 aa), alta precisión	2–4	16–32	64–128 GB
Máximo permitido	4	64	256 GB

---

4. Catálogo de proteínas disponibles

Las siguientes 22 proteínas tienen **identificación automática** y metadata real (UniProt, PDB). Cualquier otra secuencia se procesa con predicción sintética.

ID para API	Nombre	UniProt	PDB	aa	Categoría	Organismo
`ubiquitin`	Ubiquitin	P0CG47	1UBQ	76	signaling	*H. sapiens*
`histone_h4`	Histone H4	P62805	1AOI	102	structural	*H. sapiens*
`thioredoxin`	Thioredoxin	P10599	1ERU	105	enzyme	*H. sapiens*
`cytochrome_c`	Cytochrome c	P99999	1HRC	105	signaling	*H. sapiens*
`insulin_human`	Insulin	P01308	4MIF	110	hormone	*H. sapiens*
`beta2_microglobulin`	Beta-2-microglobulin	P61769	1LDS	119	immune	*H. sapiens*
`rnase_a`	Ribonuclease A	P61823	1FS3	124	enzyme	*B. taurus*
`lysozyme`	Lysozyme C (Chicken)	P61626	1LYZ	130	enzyme	*G. gallus*
`hemoglobin_alpha`	Hemoglobin alpha	P69905	1A3N	141	transport	*H. sapiens*
`hemoglobin_beta`	Hemoglobin beta	P68871	1A3N	147	transport	*H. sapiens*
`lysozyme_human`	Lysozyme C (Human)	P00695	1LZ1	148	enzyme	*H. sapiens*
`calmodulin`	Calmodulin	P0DP23	1CLL	149	signaling	*H. sapiens*
`sod1`	Superoxide dismutase 1	P00441	1PU0	154	enzyme	*H. sapiens*
`myoglobin`	Myoglobin	P02144	1MBN	154	transport	*H. sapiens*
`adenylate_kinase`	Adenylate kinase 1	P00571	4AKE	214	enzyme	*H. sapiens*
`gfp`	Green Fluorescent Protein	P42212	1GFL	239	reporter	*A. victoria*
`chymotrypsin`	Chymotrypsin C	P06431	1CHY	245	enzyme	*B. taurus*
`pcna`	PCNA	P12004	1AXC	261	dna-replication	*H. sapiens*
`p53`	Tumor protein p53	P04637	2OCJ	393	oncology	*H. sapiens*
`egfr_kinase`	EGFR kinase domain	P00533	1IVO	421	oncology	*H. sapiens*
`amylase`	Pancreatic alpha-amylase	P04746	1BVN	511	enzyme	*H. sapiens*

ID para API	Nombre	UniProt	PDB	aa	Categoría	Organismo
`serum_albumin`	Human serum albumin	P02768	1AO6	585	transport	*H. sapiens*

Para ver detalles de cualquiera:

```
curl http://localhost:8000/proteins/calmodulin
```

---

5. Secuencias FASTA listas para copiar

> **Recomendación:** La forma más fiable de obtener la secuencia exacta de cualquier proteína es llamar al endpoint:
>
> ```
> curl https://api-mock-cesga.onrender.com/proteins/ubiquitin
> ```
>
> El campo `fasta_ready` del response es exactamente el valor que tienes que pasar como `fasta_sequence`. Así garantizas que la proteína será identificada y obtendrás `protein_metadata` real en los outputs.

Las secuencias de abajo son copy-paste directamente funcionales para las proteínas más usadas:

Ubiquitin (76 aa) — la más pequeña, ideal para pruebas rápidas

```
>sp|P0CG47|UBQ_HUMAN Ubiquitin OS=Homo sapiens
MQIFVKTLTGKTITLEVEPSDTIENVKAKIQDKEGIPPDQQRLIFAGKQLEDGRTLSDYNIQKESTLHLVLRLR
GG
```

Calmodulin (149 aa) — proteína de señalización de calcio

```
>sp|P0DP23|CALM1_HUMAN Calmodulin-1 OS=Homo sapiens
MADQLTEEQIAEFKEAFSLFDKDGDGTITTKELGTVMRSLGQNPTEAELQDMINEVDADGNGTIDFPEFLTMMA
RKMKDTDSEEEIREAFRVFDKDGNGYISAAELRHVMTNLGEKLTDEEVDEMIREADIDGDGQVNYEEFVQMMTA
K
```

Histone H4 (102 aa) — componente del nucleosoma

```
>sp|P62805|H4_HUMAN Histone H4 OS=Homo sapiens
SGRGKGGKGLGKGGAKRHRKVLRDNIQGITKPAIRRLARRGGVKRISGLIYEETRGVLKVFLENVIRDAVTYTE
HAKRKTVTAMDVVYALKRQGRTLYGFGG
```

Cytochrome c (105 aa) — transporte de electrones mitocondrial

```
>sp|P99999|CYC_HUMAN Cytochrome c OS=Homo sapiens
MGDVEKGKKIFIMKCSQCHTVEKGGKHKTGPNLHGLFGRKTGQAPGYSYTAANKNKGIIWGEDTLMEYLENPKK
YIPGTKMIFVGIKKKEERADLIAYLKKATNE
```

Thioredoxin (105 aa) — defensa antioxidante

```
>sp|P10599|THIO_HUMAN Thioredoxin OS=Homo sapiens
MVKQIESKTAFQEALDAAGDKLVVVDFSATWCGPCRMIAPILDEIADEYQGKLTVAKLNIDQNPGTAPKYGIRG
IPTLLLFKNGEVAATKVGALSKGQLKEFLDANLA
```

Insulin (110 aa) — hormona pancreática

```
>sp|P01308|INS_HUMAN Insulin OS=Homo sapiens
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGQVELGGGPGA
GSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN
```

SOD1 (154 aa) — enzima antioxidante, mutada en ELA

```
>sp|P00441|SODC_HUMAN Superoxide dismutase [Cu-Zn] OS=Homo sapiens
MATKAVCVLKGDGPVQGIINFEQKESNGPVKVWGSIKGLTEGLHGFHVHEFGDNTAGCTSAGPHFNPLSRKHGG
PKDEERHVGDLGNVTADKDGVADVSIEDSVISLSGDHCIIGRTLVVHEKADDLGKGGNEESTKTGNAGSRLACG
VIGIAQ
```

Myoglobin (154 aa) — almacenamiento de oxígeno muscular

```
>sp|P02144|MYG_HUMAN Myoglobin OS=Homo sapiens
MGLSDGEWQQVLNVWGKVEADIPGHGQEVLIRLFKGHPETLEKFDKFKHLKTEAEMKASED
```

Hemoglobin alpha (141 aa) — transportador de O_2

```
>sp|P69905|HBA_HUMAN Hemoglobin subunit alpha OS=Homo sapiens
MVLSPADKTNVKAAWGKVGAHAGEYGAEALERMFLSFPTTKTYFPHFDLSHGSAQVKGHG
```

Hemoglobin beta (147 aa) — mutada en anemia drepanocítica

```
>sp|P68871|HBB_HUMAN Hemoglobin subunit beta OS=Homo sapiens
MVHLTPEEKSAVTALWGKVNVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDAVMGNPKVKAHGKKVLGAFSD
GLAHLDNLKGTFATLSELHCDKLHVDPENFRLLGNVLVCVLAHHFGKEFTPPVQAAYQKVVAGVANALAHKYH
```

GFP (239 aa) — proteína fluorescente verde

```
>sp|P42212|GFP_AEQVI Green fluorescent protein OS=Aequorea victoria
MSKGEELFTGVVPILVELDGDVNGHKFSVSGEGEGDATYGKLTLKFICTTGKLPVPWPTLVTTLTYGVQCFSRY
PDHMKQHDFFKSAMPEGYVQERTIFFKDDGNYKTRAEVKFEGDTLVNRIELKGIDFKEDGNILGHKLEYNYNSH
NVYIMADKQKNGIKVNFKIRHNIEDGSVQLADHYQQNTPIGDGPVLLPDNHYLSTQSALSKDPNEKRDHMVLLE
FVTAAGITLGMDELYK
```

Lysozyme chicken (130 aa) — proteína modelo clásica

```
>sp|P61626|LYSC_CHICK Lysozyme C OS=Gallus gallus
MRSLLILVVTFLAGCSAKAKDQGNLSGAEKAVQVKVKALPDAQFEVVHSLAKWKRQTLGQHDFSAGEGLYTHMK
ALRPDEDRLSPLHSVYVDQWDWERVMGDGERQFSTLKSTVEAIWAGIKATEAAVSEEFGLAPFLPDQIHFVHSQ
ELLSRYPDLDAKGRERAIAKDLGAVFLVGIGGKLSDGHRHDVRAPDYDDWSNPSELGHAFRNGYRTTDVTNRFT
GVVTADTSKDKAAQGFTVQREVSPYSDVQAKD
```

PCNA (261 aa) — factor de processividad de ADN polimerasa

```
>sp|P12004|PCNA_HUMAN Proliferating cell nuclear antigen OS=Homo sapiens
MFEARLVQGSILKKVLEALKDLINEACWDISSSGVNLQDLGITAIEGFETLKVDLDASLNIKLTNERFLKQDNV
HVLMCDKSDKIRKKLGEELDSRQETLVLGSIESLASLIEDIFQSRLEDLNQMVSKIQVYMSDFKTKQKCLDALQ
KFLEESED
```

p53 (393 aa) — supresor tumoral "guardián del genoma"

```
>sp|P04637|P53_HUMAN Cellular tumor antigen p53 OS=Homo sapiens
MEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLPSQAMDDLMLSPDDIEQWFTEDPGPDEAPRMPEAAPPVA
PAPAAPTPAAPAPAPSWPLSSSVPSQKTYPQGLASPSNMDDLMLSPDDIEQWFTEDPGP
```

---

6. Entendiendo los outputs: pLDDT y PAE

pLDDT — Confianza local por residuo

pLDDT (predicted Local Distance Difference Test) es el indicador de confianza de AlphaFold2. Va de 0 a 100.

Rango pLDDT	Color en AlphaFold DB	Interpretación
> 90	Azul oscuro	**Muy alta confianza** — estructura casi segura

Rango pLDDT	Color en AlphaFold DB	Interpretación
70–90	Azul claro	**Alta confianza** — generalmente correcta
50–70	Amarillo	**Confianza baja** — tomar con cautela
< 50	Naranja	**Muy baja confianza** — región desordenada probablemente

En el response encontrarás:

```
"confidence": {
  "plddt_per_residue": [72.3, 85.1, 91.4, 45.2, ...],  // Un valor por re
  "plddt_mean": 71.7,                                   // Media global
  "plddt_histogram": {
    "very_high": 13,   // Residuos con pLDDT > 90
    "high": 29,        // Residuos con pLDDT 70-90
    "medium": 24,      // Residuos con pLDDT 50-70
    "low": 10          // Residuos con pLDDT < 50
  }
}
```

Regla práctica: Si `plddt_mean > 70`, la estructura global es fiable. Si hay muchos residuos `low`, pueden ser regiones intrínsecamente desordenadas (IDRs).

PAE — Error de alineación predicho (Predicted Aligned Error)

La **PAE matrix** es una matriz NxN (N = número de residuos). Cada celda `[i][j]` indica el error esperado en Ångströms entre los residuos i y j.

- **Valor bajo (~0–5 Å):** Alta confianza en la posición relativa entre esos dos residuos
- **Valor alto (>10 Å):** Los dos residuos tienen posición relativa incierta entre sí

```
"pae_matrix": [
  [0.1, 0.8, 1.2, ...],   // fila i=0: distancias desde residuo 0 al rest
  [0.8, 0.1, 0.9, ...],   // fila i=1: distancias desde residuo 1 al rest
  ...
],
"mean_pae": 3.33           // Media global
```

Uso en frontend: Renderiza la PAE matrix como un heatmap 2D. Bloques de color azul (bajo error) en off-diagonal indican **dominios bien definidos** con orientación relativa confiable. Bloques amarillos/rojos entre dominios indican **flexibilidad o desorientación inter-dominios**.

Datos biológicos — `biological_data`

Campo	Rango	Interpretación
`solubility_score`	0–100	>50: soluble; <30: probablemente insoluble en agua
`instability_index`	0–100	<40: proteína estable (vida media larga); >40: inestable
`toxicity_alerts`	lista	Motivos de secuencia con riesgo potencial
`allergenicity_alerts`	lista	Características asociadas a alergenicidad
`secondary_structure_prediction.helix_percent`	0–100	% de residuos en alfa-hélice estimado
`secondary_structure_prediction.strand_percent`	0–100	% de residuos en lámina beta estimado
`secondary_structure_prediction.coil_percent`	0–100	% restante en coil/loop

Accounting — Contabilidad HPC

Campo	Unidades	Descripción
`cpu_hours`	CPU·h	Tiempo de CPU consumido
`gpu_hours`	GPU·h	Tiempo de GPU consumido (0 si sin GPU)
`memory_gb_hours`	GB·h	Memoria×tiempo consumida
`total_wall_time_seconds`	segundos	Tiempo real de ejecución
`cpu_efficiency_percent`	%	Uso real / uso máximo posible de CPU
`memory_efficiency_percent`	%	Uso real / memoria solicitada
`gpu_efficiency_percent`	%	Ocupación GPU durante la ejecución

---

7. Errores comunes y cómo resolverlos

422 Unprocessable Entity — error de validación

La request tiene un campo inválido.

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": ["body", "fasta_sequence"],
      "msg": "Value error, FASTA sequence must start with '>'"
    }
  ]
}
```

Causas frecuentes:

Error	Causa	Solución
`FASTA sequence must start with '>'`	Falta el header FASTA	Añade `>nombre_proteina\n` antes de la secuencia
`Input should be less than or equal to 4`	GPUs > 4	Usa máximo 4 GPUs
`Input should be less than or equal to 256`	Memoria > 256 GB	Máximo 256 GB
`Input should be greater than or equal to 60`	`max_runtime_seconds` < 60	Mínimo 60 segundos

---

400 Bad Request — outputs no disponibles aún

```
{
  "detail": "Job outputs only available when status=COMPLETED. Current st
}
```

Solución: Espera a que el estado sea `COMPLETED` antes de llamar a `/outputs`. El tiempo típico es ~10 segundos.

---

404 Not Found — job o proteína no existe

```
{"detail": "Job job_xyz not found"}
{"detail": "Protein 'foo' not found. Use GET /proteins to see all availab
```

---

500 Internal Server Error

Si aparece un 500 inesperado, el body incluirá el error detallado:

```
{"detail": "Internal server error", "error": "...mensaje..."}
```

---

8. Flujo completo de ejemplo

Opción A: con curl (terminal)

```
BASE="https://api-mock-cesga.onrender.com"

# 1. Ver qué proteínas hay disponibles
curl "$BASE/proteins/"

# 2. Obtener la secuencia de calmodulin lista para usar
curl "$BASE/proteins/calmodulin" | python3 -m json.tool

# 3. Enviar el job
JOB=$(curl -s -X POST "$BASE/jobs/submit" \
  -H "Content-Type: application/json" \
  -d '{
    "fasta_sequence": ">sp|P0DP23|CALM1_HUMAN Calmodulin-1\nMADQLTEEQIAEF
    "fasta_filename": "calmodulin.fasta",
    "gpus": 1,
    "cpus": 8,
    "memory_gb": 32.0
  }')
echo $JOB
JOB_ID=$(echo $JOB | python3 -c "import json,sys; print(json.load(sys.stc

# 4. Esperar a que complete (poll cada 2 segundos)
while true; do
  STATUS=$(curl -s "$BASE/jobs/$JOB_ID/status" | python3 -c "import json,
  echo "Estado: $STATUS"
  if [ "$STATUS" = "COMPLETED" ] || [ "$STATUS" = "FAILED" ]; then break;
  sleep 2
done

# 5. Obtener outputs
```

```
curl -s "$BASE/jobs/$JOB_ID/outputs" | python3 -m json.tool > resultados.

# 6. Extraer el fichero PDB para visualizar en PyMOL / Mol*
python3 -c "
import json
with open('resultados.json') as f:
    d = json.load(f)
pdb = d['structural_data']['pdb_file']
with open('calmodulin.pdb', 'w') as f:
    f.write(pdb)
print('Guardado: calmodulin.pdb')
print('pLDDT medio:', d['structural_data']['confidence']['plddt_mean'])
print('Solubilidad:', d['biological_data']['solubility_score'])
"

# 7. Contabilidad de recursos
curl "$BASE/jobs/$JOB_ID/accounting" | python3 -m json.tool
```

Opción B: con Python (requests)

```
import requests
import time
import json

BASE_URL = "https://api-mock-cesga.onrender.com"

# 1. Ver proteínas disponibles
proteins = requests.get(f"{BASE_URL}/proteins/").json()
print(f"Proteínas en catálogo: {len(proteins)}")
for p in proteins[:5]:
    print(f"  {p['protein_id']:20s} | {p['length']} aa | {p['protein_name

# 2. Obtener secuencia exacta de ubiquitin (fasta_ready es el campo que h
ubiquitin = requests.get(f"{BASE_URL}/proteins/ubiquitin").json()
fasta = ubiquitin["fasta_ready"]
print("\nFASTA listo:", fasta[:60])

# 3. Enviar job (campos: fasta_sequence, fasta_filename, gpus, cpus, memo
response = requests.post(f"{BASE_URL}/jobs/submit", json={
    "fasta_sequence": fasta,
    "fasta_filename": "ubiquitin.fasta",
    "gpus": 1,
```

```
    "cpus": 8,
    "memory_gb": 32.0,
    "max_runtime_seconds": 3600,
})
response.raise_for_status()
job = response.json()
job_id = job["job_id"]
print(f"\nJob creado: {job_id}")

# 4. Esperar a COMPLETED
while True:
    status_resp = requests.get(f"{BASE_URL}/jobs/{job_id}/status").json()
    status = status_resp["status"]
    print(f"  Estado: {status}")
    if status in ("COMPLETED", "FAILED", "CANCELLED"):
        break
    time.sleep(2)

# 5. Obtener outputs
if status == "COMPLETED":
    outputs = requests.get(f"{BASE_URL}/jobs/{job_id}/outputs").json()

    # Guardar PDB
    pdb_content = outputs["structural_data"]["pdb_file"]
    with open("structure.pdb", "w") as f:
        f.write(pdb_content)

    # Ver confianza
    conf = outputs["structural_data"]["confidence"]
    print(f"\npLDDT medio: {conf['plddt_mean']:.1f}")
    print(f"Histograma: {conf['plddt_histogram']}")

    # Ver datos biológicos
    bio = outputs["biological_data"]
    print(f"Solubilidad: {bio['solubility_score']:.1f} ({bio['solubility_
    print(f"Inestabilidad: {bio['instability_index']:.1f} ({bio['stabilit

    # Metadata de la proteína identificada
    meta = outputs.get("protein_metadata")
    if meta:
        print(f"\nProteína identificada: {meta['protein_name']}")
        print(f"UniProt: {meta['uniprot_id']} | PDB: {meta['pdb_id']}")

    # Contabilidad
```

```
    acc = requests.get(f"{BASE_URL}/jobs/{job_id}/accounting").json()
    accounting = acc["accounting"]
    print(f"\nRecursos usados:")
    print(f"   CPU-hours:  {accounting['cpu_hours']:.4f}")
    print(f"   GPU-hours:  {accounting['gpu_hours']:.4f}")
    print(f"   Wall time:  {accounting['total_wall_time_seconds']} s")
```

Opción C: con JavaScript / fetch

```
const BASE_URL = "https://api-mock-cesga.onrender.com";

async function runFullWorkflow() {
  // 1. Enviar job (campos obligatorios: fasta_sequence, fasta_filename)
  const submitResp = await fetch(`${BASE_URL}/jobs/submit`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      fasta_sequence: ">sp|P0CG47|UBQ_HUMAN Ubiquitin\nMQIFVKTLTGKTITLEVE
      fasta_filename: "ubiquitin.fasta",
      gpus: 1,
      cpus: 8,
      memory_gb: 32.0,
    }),
  });
  const { job_id } = await submitResp.json();
  console.log("Job ID:", job_id);

  // 2. Polling hasta COMPLETED
  let status = "PENDING";
  while (!["COMPLETED", "FAILED", "CANCELLED"].includes(status)) {
    await new Promise(r => setTimeout(r, 2000));
    const st = await fetch(`${BASE_URL}/jobs/${job_id}/status`).then(r =>
    status = st.status;
    console.log("Estado:", status);
  }

  // 3. Obtener outputs
  if (status === "COMPLETED") {
    const outputs = await fetch(`${BASE_URL}/jobs/${job_id}/outputs`).the

    const pdbContent = outputs.structural_data.pdb_file;
    const plddt = outputs.structural_data.confidence.plddt_mean;
```

```
    const bio = outputs.biological_data;

    console.log(`pLDDT medio: ${plddt.toFixed(1)}`);
    console.log(`Solubilidad: ${bio.solubility_score.toFixed(1)}`);

    // Si tu frontend usa Mol* o 3Dmol.js, pasa pdbContent directamente a
    // viewer.loadStructureFromData(pdbContent, "pdb");
  }
}

runFullWorkflow();
```

Apéndice: Referencia rápida de endpoints

```
GET  /                          →  Información de la API
GET  /health                    →  Estado del servidor
GET  /docs                      →  Swagger UI interactivo

POST /jobs/submit               →  Enviar nuevo job
GET  /jobs/                     →  Listar todos los jobs
GET  /jobs/{id}/status          →  Estado de un job
GET  /jobs/{id}/outputs         →  Resultados (solo COMPLETED)
GET  /jobs/{id}/accounting      →  Contabilidad de recursos HPC

GET  /proteins/                 →  Listar proteínas del catálogo
GET  /proteins/stats            →  Estadísticas de la base de datos
GET  /proteins/samples          →  Secuencias de ejemplo listas
GET  /proteins/{protein_id}     →  Detalles + FASTA de una proteína
```

Campos de `POST /jobs/submit` — resumen rápido

Campo	Requerido	Tipo	Notas
`fasta_sequence`	✅	string	Debe empezar con `>`
`fasta_filename`	✅	string	p.ej. `mi_proteina.fasta`
`gpus`	❌	int (0–4)	Default: 0
`cpus`	❌	int (1–64)	Default: 1
`memory_gb`	❌	float (0–256)	Default: 4.0

Campo	Requerido	Tipo	Notas
`max_runtime_seconds`	❌	int (60–86400)	Default: 3600

> ⚠️ No existe un campo `job_name` ni `num_gpus`. Usar `gpus` para las GPUs.